ECEN 4/5632 Theory and Application of Digital Filters

Lecture 14, 10-27-26

Peter Mathys*



* With generous help from Gemini Pro and ChatGPT

WEEK 8 ● LECTURE 14

Upsampling, Interpolation, and Rational Rate Changes

From zero insertion to sample-rate conversion and transposed convolutions



Oppenheim & Schafer §4.6

ROADMAP

Learning objectives

The goal is to design complete sample-rate conversion chains and connect them to learned upsampling.

- Construct an upsampled sequence by zero insertion.
- Explain why the DTFT becomes X(e^{jωL}) and why spectral images appear.
- Choose and scale an interpolation lowpass filter.
- Build an L/M rational rate converter and select its cutoff.
- Relate ConvTranspose1d to zero insertion + convolution and diagnose periodic artifacts.

learned upsampling in ML

Pacing

0–20 min: upsampling
20–45 min: interpolation
45–60 min: L/M conversion
60–75 min: transposed conv

Core question

After inserting zeros, what must the filter do in both frequency and amplitude?

01

Upsampling by zero insertion

Create a higher-rate sequence by inserting L−1 zeros between samples.

Upsampling: the time-domain definition

$$x_i[n] = \begin{cases} x[n/L], & n = 0, \pm L, \pm 2L, \dots \\ 0, & \text{otherwise} \end{cases}$$

- Output sample rate becomes L F_s.
- Original samples remain at indices n = kL.
- Inserted zeros are placeholders—not estimates of the missing waveform.
- Interpolation filtering will fill in meaningful intermediate values.

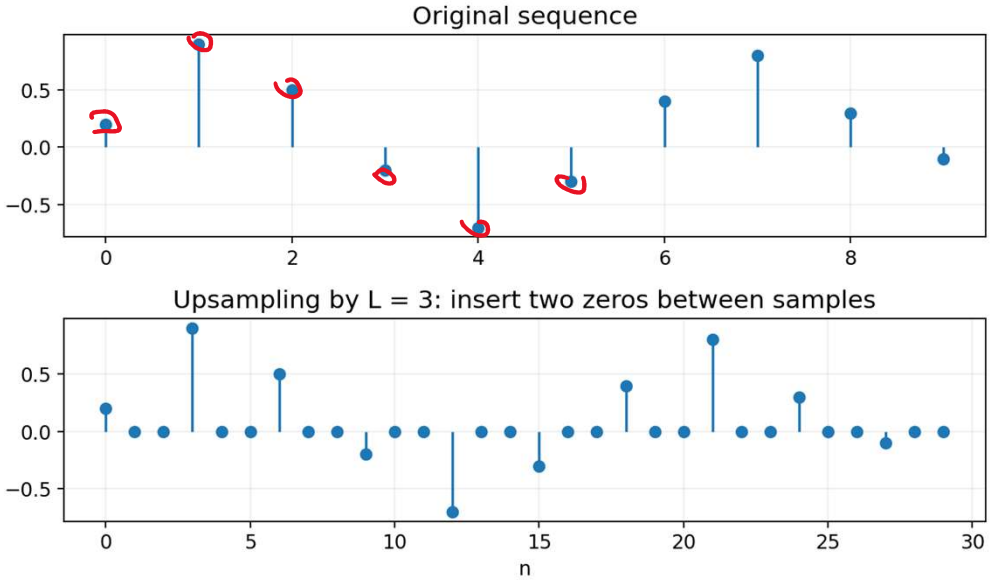
$$x_i[n] = x[n/L] \sum_k \delta[n - kL]$$

$$= \sum_k x\left[\frac{kL}{L}\right] \cdot \delta[n - kL]$$

sifting prop of $\delta[n]$

$= x[k]$

L = 3



finer grid in time domain

Interpolated $x[n]$:

$$x_i[n] = \sum_k x[k] \cdot \delta[n - kL]$$

z-transform

$$X_i(z) = \sum_n x_i[n] z^{-n} = \sum_n \sum_k x[k] \cdot \delta[n - kL] \cdot z^{-n}$$

$$= \sum_n \sum_k x[k] z^{-kL} \cdot \delta[n - kL]$$

$$= \sum_k x[k] z^{-kL} \underbrace{\sum_n \delta[n - kL]}_{= 1} = \sum_k x[k] z^{-kL} = X(z^L)$$

$$\Rightarrow X_i(e^{j\omega}) = X(e^{j\omega L})$$

upsampled original

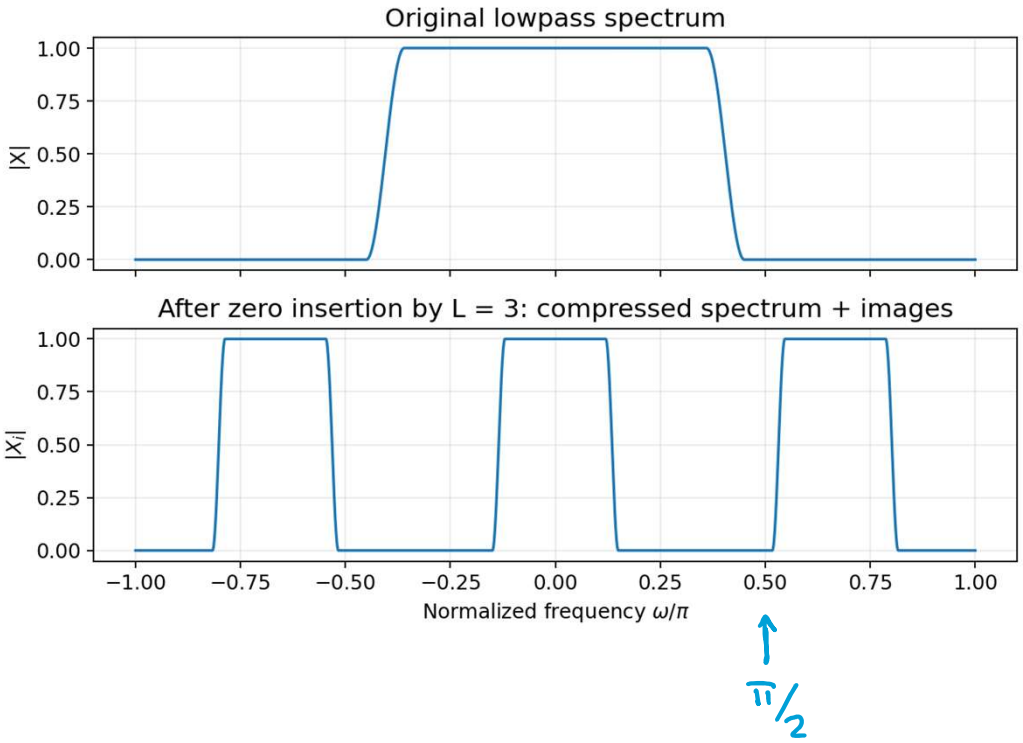
L upsampling factor

0:08–0:15

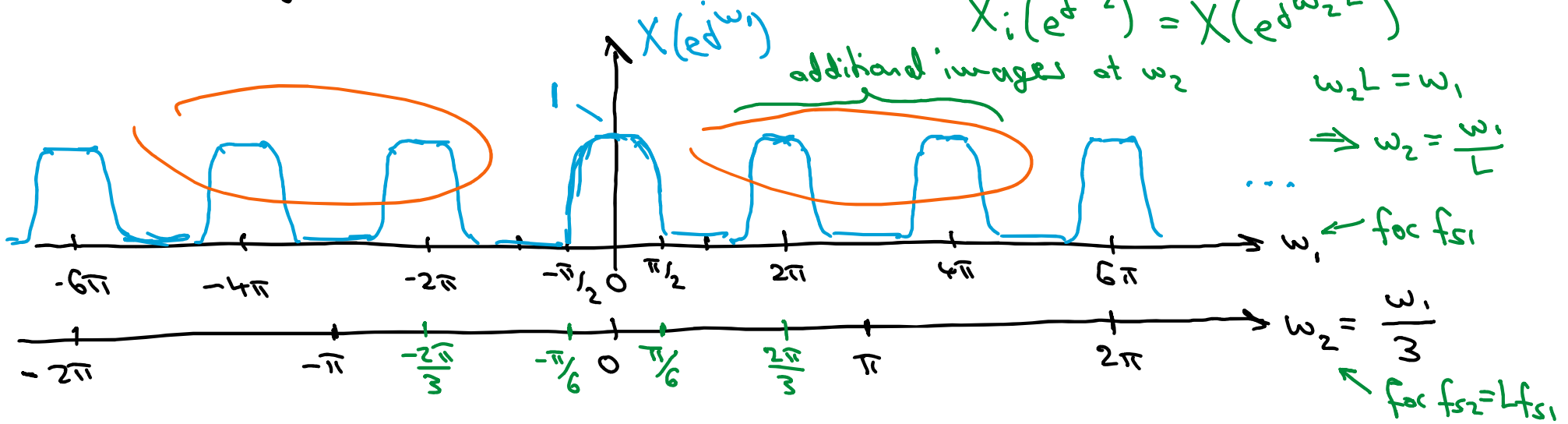
Frequency-domain effect: compression + images

$$X_i(e^{j\omega}) = X(e^{j\omega L})$$

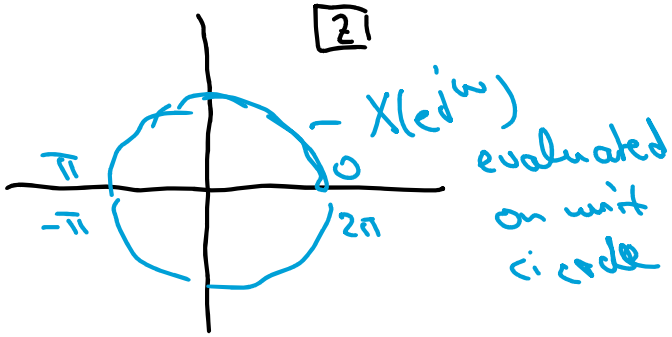
- **Compression:** the original spectrum is squeezed by L.
- **Periodicity:** the compressed copy repeats L times over [−π, π].
- **Images:** the extra replicas are artifacts of zero insertion.
- **Goal:** keep the baseband copy; remove the rest.



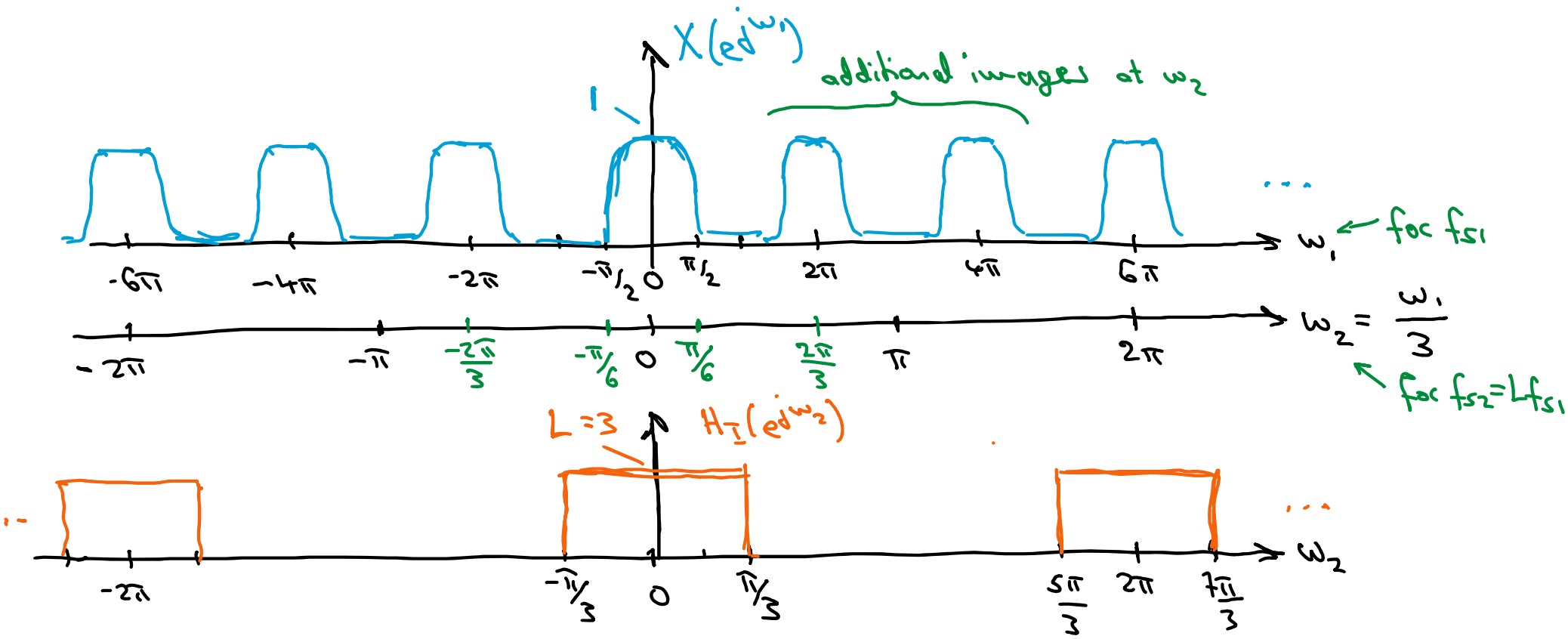
Upsampling by $L=3$ in freq domain



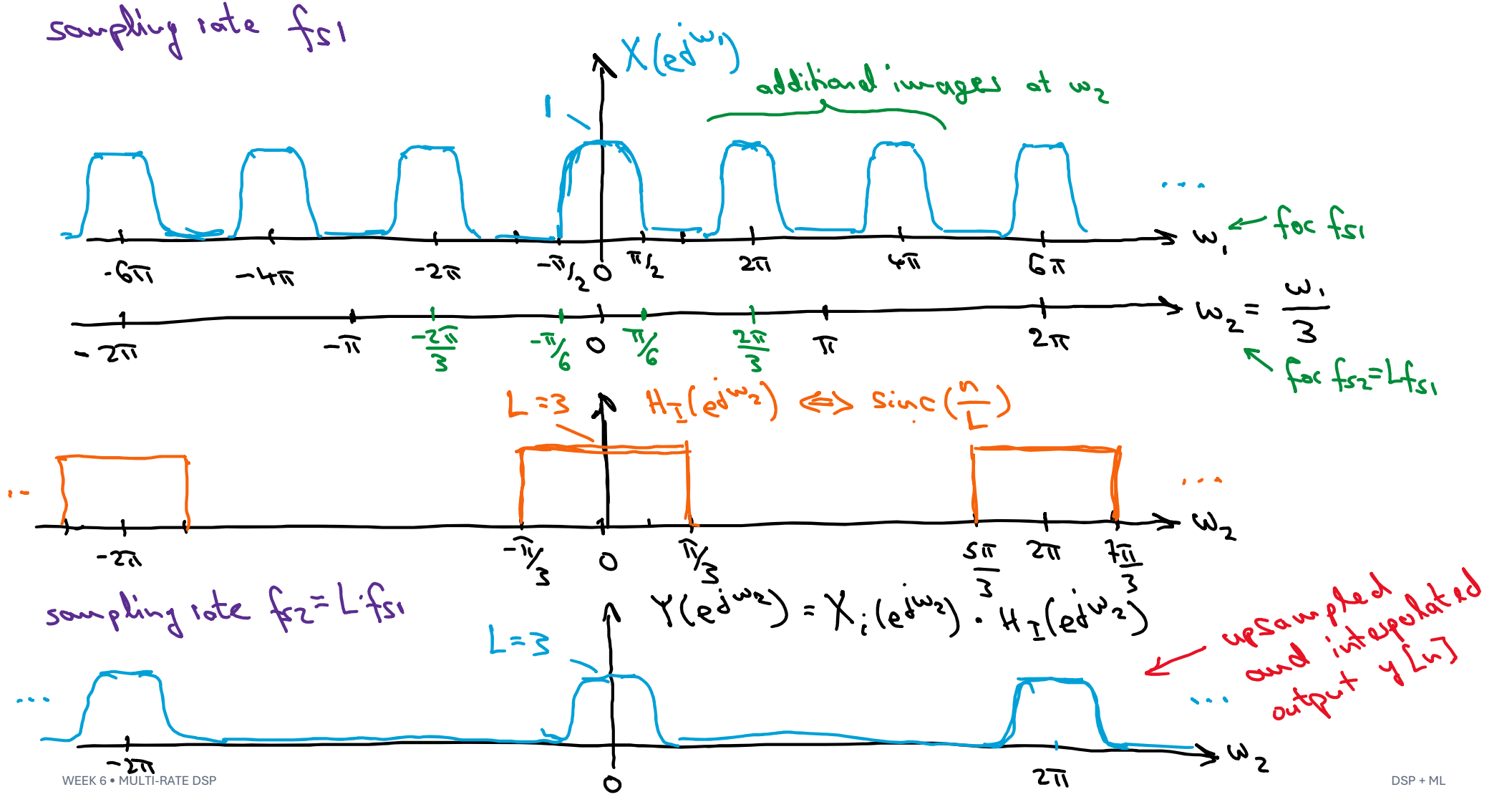
[2]



we need filter to eliminate images after upsampling by L



$H_I(e^{j\omega_2})$ is interpolation filter to suppress the additional images from upsampling


sampling rate f_{s1}
$X(e^{j\omega_1})$
additional images at ω_2
ω_1
for f_{s1}
$\omega_2 = \frac{\omega_1}{3}$
for $f_{s2} = L f_{s1}$
$L=3$
$H_I(e^{j\omega_2}) \Leftrightarrow \text{sinc}\left(\frac{n}{L}\right)$
ω_2
sampling rate $f_{s2} = L \cdot f_{s1}$
$L=3$
$Y(e^{j\omega_2}) = X_i(e^{j\omega_2}) \cdot H_I(e^{j\omega_2})$
upsampled and interpolated output $y[n]$
ω_2

Concept check: image locations

If L = 3 and the original signal is baseband, how many compressed spectral copies appear over one 2π interval after zero insertion?

Reveal after discussion

for general L we get L copies because we go around unit circle L times after upsampling

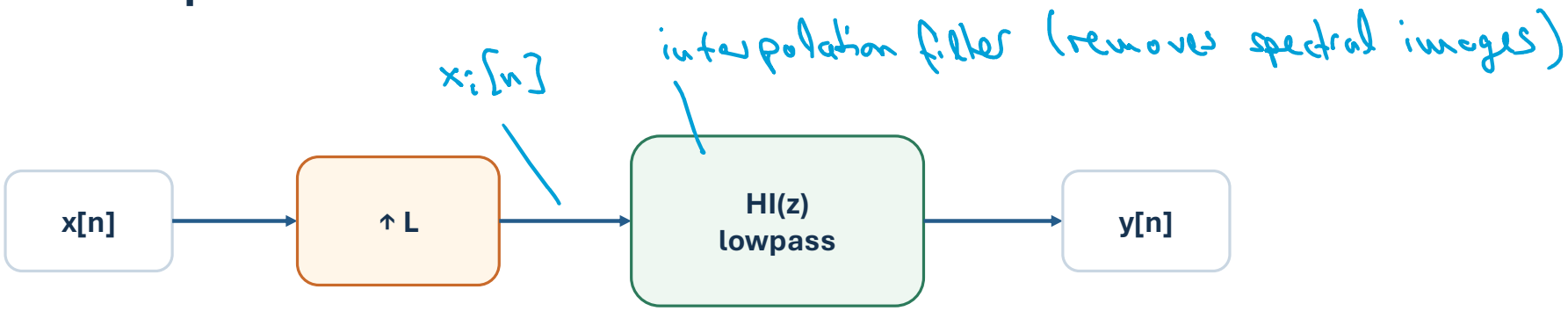
02

Interpolation = upsample + lowpass

The anti-imaging filter isolates the desired baseband replica and reconstructs intermediate samples.

0:20–0:28

The interpolation filter



Ideal cutoff

$\omega_c = \pi/L$

Pass only one image

The lowpass retains the compressed baseband copy centered at $\omega = 0$.

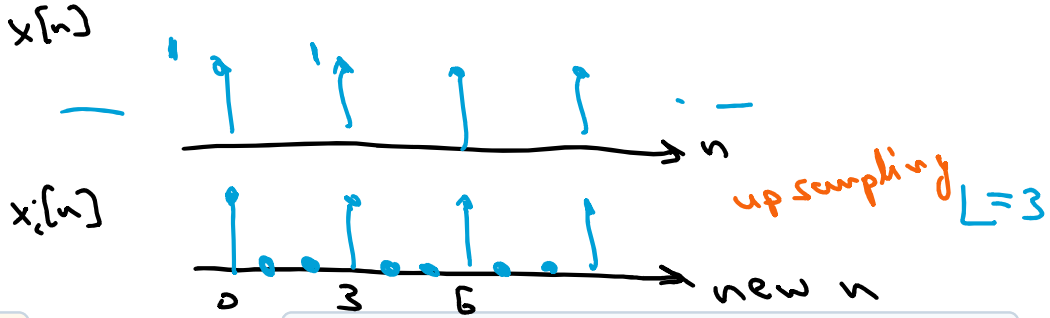
DC gain

Set $H_I(e^{j0}) = L$ so the reconstructed sequence preserves amplitude.

0:28–0:34

Why the gain must be L

Test signal: x[n] = 1 for all n.

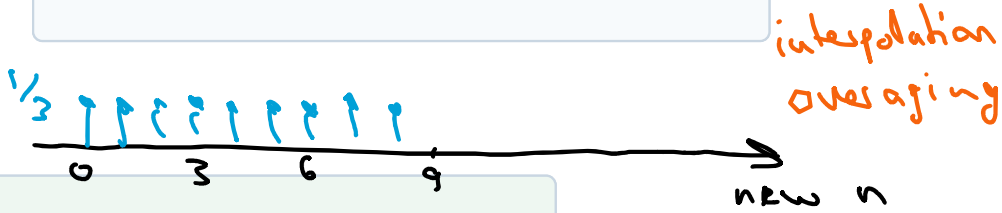


After ↑L

Sequence is 1, 0, 0, …, 1, 0, 0, …
Average value = 1/L

Lowpass with unit DC gain

Smoothing preserves the reduced average → output tends toward 1/L.

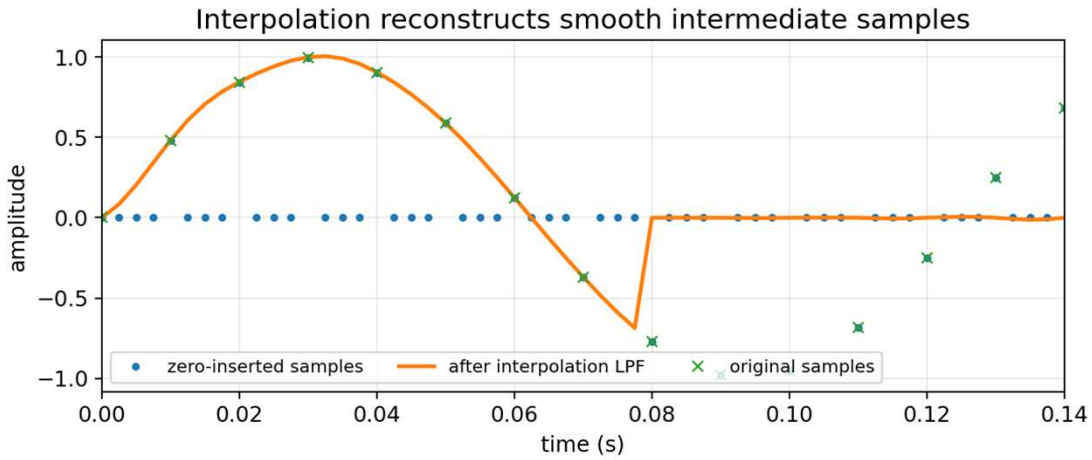


Correct interpolation filter

Multiply the lowpass by L so constant input stays constant: y[n] = 1.

0:34–0:40

Interpolation in the time domain



- Inserted zeros disappear after filtering.
- Original sample locations are retained (subject to filter delay / edge effects).
- Intermediate samples are weighted combinations of neighboring originals.
- Ideal sinc interpolation is conceptually exact for bandlimited signals; FIR approximations are practical.

DSP interpretation

The filter is reconstructing a smoother sequence on a denser sampling grid.

0:40–0:45

Choosing an interpolation FIR

Passband

Preserve the compressed baseband: $|\omega| \le \omega_p < \pi/L$.

Stopband

Suppress the first image beginning near π/L.

Gain

Scale coefficients so the DC gain is L.

Delay

For N-tap linear-phase FIR, delay = $(N-1)/2$ high-rate samples.

Implementation check: feed a constant signal and verify the output amplitude is unchanged away from transients.

03

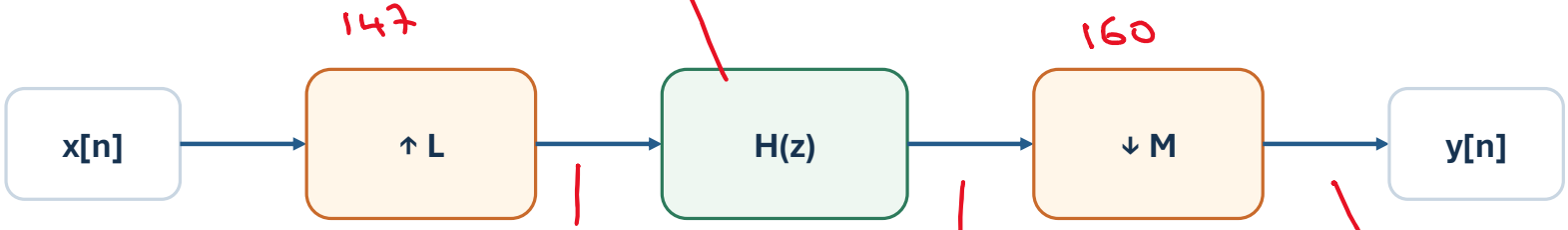
Rational sample-rate conversion L/M

Upsample first, filter once, then downsample.

0:45–0:51

Why the order is ↑L then ↓M

$$\frac{L}{M} = \frac{L/\gcd(L,M)}{M/\gcd(L,M)}$$



$f_{s_1} = 44100$

this is not correct, see next slide

$f_s = 147 \cdot 44100$

$f_s = 147 \cdot 44100$

$f_{s_2} = \frac{147}{160} \cdot 44100 \rightarrow f_{s_2} = 48000$

$$F_{s,out} = \frac{L}{M} F_{s,in}$$

$$\frac{44100}{48000} = \frac{147}{160}$$

- Upsampling first makes the intermediate grid fine enough to represent the desired output sample times.
- A single lowpass filter can remove interpolation images and prevent later decimation aliasing.
- Reduce L/M to lowest terms before implementation.

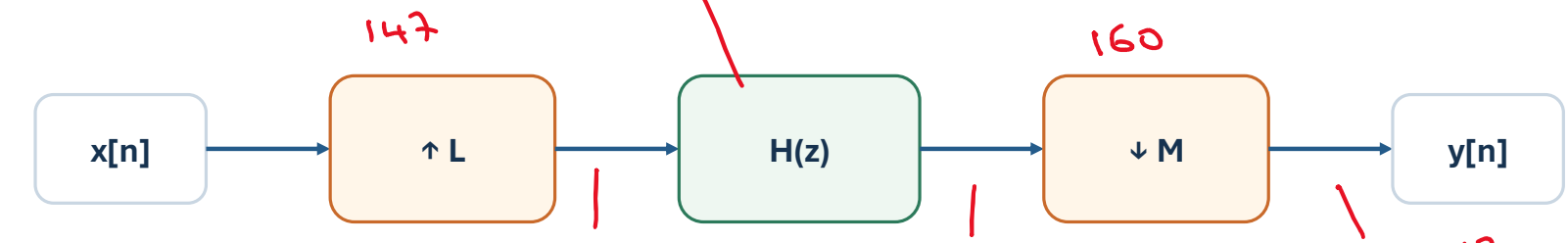
e.g $F_{s,out} = 44100$ kHz , $F_{s,in} = 48000$ kHz

0:45–0:51

Corrected Slide

Why the order is ↑L then ↓M

$$\frac{L}{M} = \frac{L/\gcd(L,M)}{M/\gcd(L,M)}$$



$f_{s,1} = 48000$

$f_s = 147 \times 48000$

$f_s = 147 \times 48000$

$f_{s2} = \frac{147}{160} \cdot 48000 \rightarrow f_{s2} = 44100$

$$F_{s,out} = \frac{L}{M} F_{s,in}$$

$$\frac{44100}{48000} = \frac{147}{160}$$

- Upsampling first makes the intermediate grid fine enough to represent the desired output sample times.
- A single lowpass filter can remove interpolation images and prevent later decimation aliasing.
- Reduce L/M to lowest terms before implementation.

e.g $F_{s,out} = 44100$ kHz , $F_{s,in} = 48000$ kHz

0:51–0:56

One filter serves two jobs

image rejection

anti-aliasing

$$\omega_c = \min\left(\frac{\pi}{L}, \frac{\pi}{M}\right)$$

Gain = L

Anti-imaging constraint

After ↑L, reject replicas outside the desired compressed baseband.

Anti-aliasing constraint

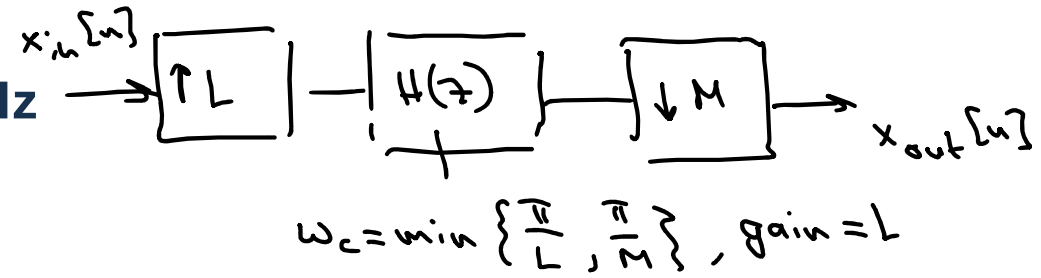
Before ↓M, remove content that lies above the new Nyquist limit.

Combined design

Use the stricter cutoff. In practice, reserve a transition band below the ideal boundary.

0:56–1:00

Real example: 48 kHz → 44.1 kHz



Rate ratio

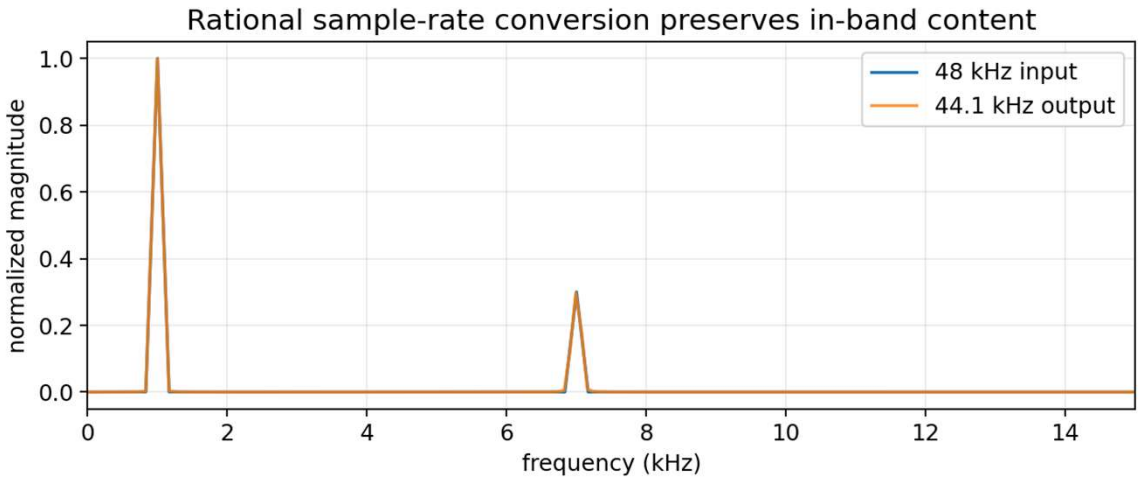
44,100 / 48,000 = 147 / 160

Conceptual chain

↑147 → lowpass → ↓160

Efficient implementation

Polyphase filtering computes only the output samples that are needed.



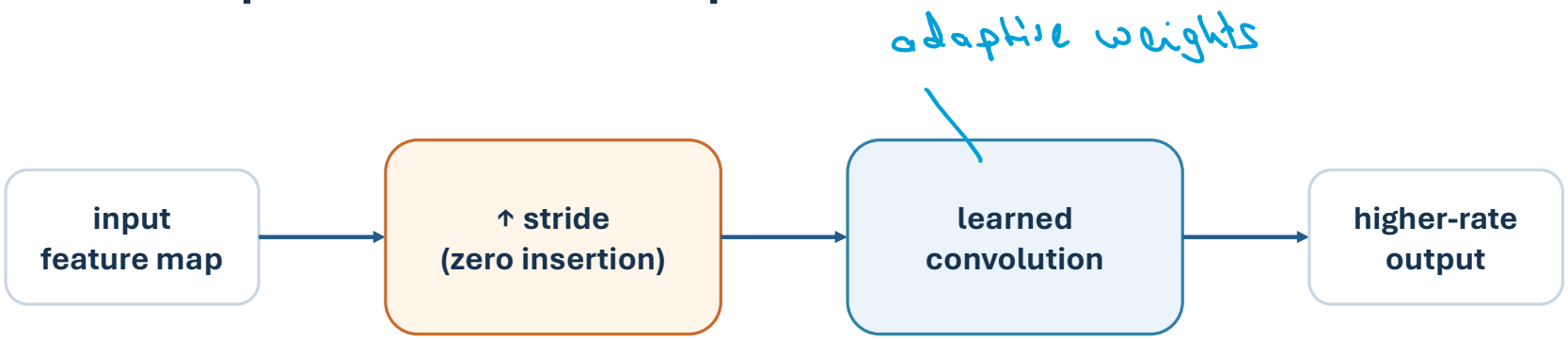
04

Transposed convolutions and learned upsampling

A stride in the reverse direction is closely related to zero insertion followed by filtering.

1:00–1:06

ConvTranspose1d: a DSP interpretation



- For stride > 1, transposed convolution can be viewed as inserting zeros and then applying a learned filter (with indexing/padding details).
- The learned filter is not automatically an anti-imaging filter.
- Uneven overlap or poorly controlled imaging can create periodic artifacts.

Stride 2 downsampling via weight matrix $\bar{W}$

Example: $w[n] = \{w_0, w_1, w_2\}$ kernel size 3

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix} = \underbrace{\begin{bmatrix} w_0 & w_1 & w_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & w_0 & w_1 & w_2 & 0 & 0 \\ 0 & 0 & 0 & 0 & w_0 & w_1 & w_2 \end{bmatrix}}_{\bar{W}} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix}$$

stride 2

instead of $y = \bar{W} \cdot x$

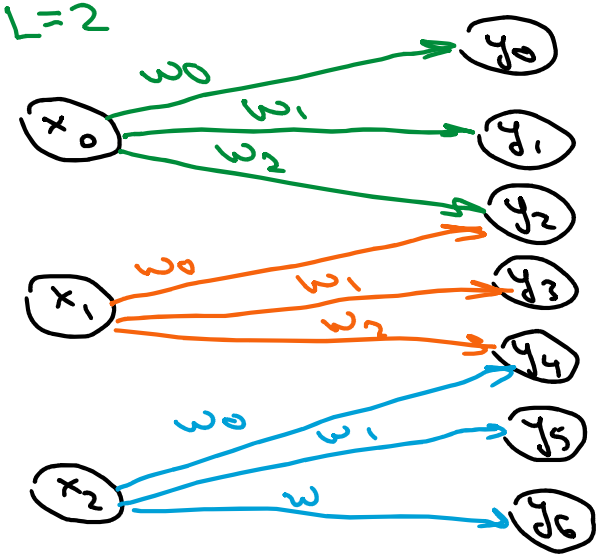
compute $y = \bar{W}^T \cdot x$ (for different x)

only $\bar{W}, \bar{W}^T$ is common here

downsampling (with filter) by $M = 2$

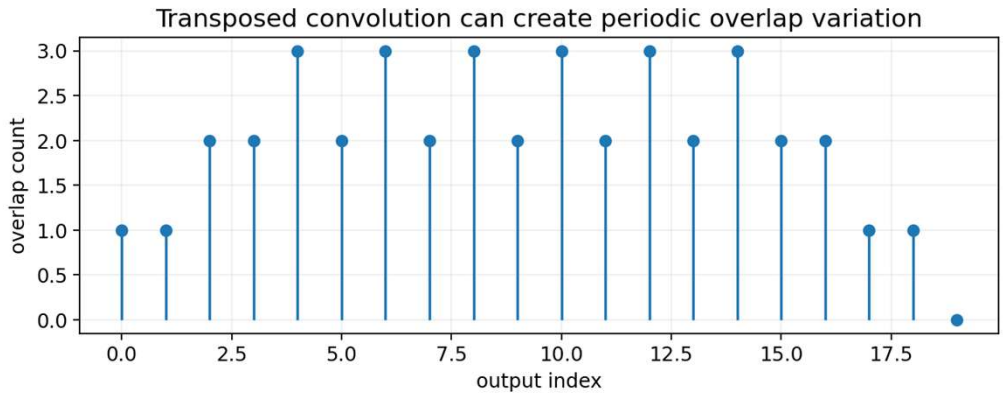
upsampling $L = 2$

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \end{bmatrix} = \begin{bmatrix} w_0 & 0 & 0 \\ w_1 & 0 & 0 \\ w_2 & w_0 & 0 \\ 0 & w_1 & 0 \\ 0 & w_2 & w_0 \\ 0 & 0 & w_1 \\ 0 & 0 & w_2 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$



1:06–1:11

Where periodic / checkerboard artifacts come from



- **Uneven overlap:** some output positions receive more kernel contributions than others.
- **Spectral images:** zero insertion creates replicas that a learned kernel may not suppress.
- **Training can exploit phase patterns:** periodic structure may become visible or audible.

Mitigation patterns

Resize/upsample + ordinary convolution, kernel/stride choices with even overlap, or explicit lowpass / anti-imaging constraints.

1:11–1:14

A DSP-informed initialization idea

Start from interpolation

Initialize each upsampling kernel with a smooth lowpass / linear-interpolation-like response.

Then learn

Allow gradient descent to adapt the kernel while regularization or architecture preserves reasonable spectral behavior.

Why it helps

- The model begins with less imaging energy.
- Early training outputs can be smoother and more stable.
- The initialization encodes a physically sensible prior without preventing learning.

1:14–1:15

Lecture 14 wrap-up

- **Upsampling** inserts zeros and creates spectral images.
- **Interpolation** lowpass filters those images and uses gain L.
- **Rational conversion** uses ↑L → LPF → ↓M with cutoff min(π/L, π/M).
- **ConvTranspose1d** is closely related to zero insertion + learned filtering.
- **DSP insight** helps explain and mitigate periodic artifacts in learned upsampling.

Exit question

Why does an interpolation filter need gain L rather than unity gain?